

Compilador de Lenguaje de Comunicación Personal

Documentación técnica completa del código fuente

Análisis Léxico · Análisis Sintáctico · Análisis Semántico

Nicolás Cisneros Alanís

nicisnerosal@ittpic.edu.mx

Lenguajes y Autómatas II · Instituto Tecnológico de Tepic

Este compilador traduce secuencias de gestos, sonidos y movimientos corporales a frases en español natural, permitiendo que el cuidador de una persona con autismo no verbal comprenda lo que desea comunicar.

1. Descripción General del Proyecto

El proyecto es un **compilador de un lenguaje de comunicación aumentativa y alternativa (CAA)**, diseñado para interpretar señales de una persona con autismo no verbal. El sistema toma como entrada una secuencia de *tokens* (gestos, sonidos, movimientos) y produce como salida una frase en español natural que describe lo que la persona intenta comunicar.

Motivación

El lenguaje fue diseñado específicamente para interpretar las señales del hermano del desarrollador, quien es no verbal. Las señales fueron observadas y categorizadas en seis grupos (E1–E6) según su naturaleza.

Arquitectura del compilador (3 fases)

Fase	Archivo	Función principal	Salida
1 · Léxico	lexer.py	Analiza carácter a carácter y divide en tokens	Lista de tokens + errores LEX-###
2 · Sintáctico	sintactico.py	Verifica que la secuencia sea gramaticalmente válida	AST (árbol de sintaxis abstracta)
3 · Semántico	semantico.py	Valida el significado y genera la frase en español	Frase en lenguaje natural + errores SEM-#

Archivos del proyecto

Archivo	Descripción
lexer.py	Define el analizador léxico con PLY
sintactico.py	Define la gramática LALR(1) y los nodos del AST
semantico.py	Análisis semántico: estructuras, validación y frases
interfaz.py	Interfaz gráfica con Tkinter
interpretador_ia.py	Módulo de IA (no activo en esta versión — reemplazado por reglas)
BITACORA.md	Registro de todos los cambios realizados al proyecto

2. Librerías Utilizadas

El proyecto usa únicamente librerías de Python. No requiere instalación de servicios externos (base de datos, servidores, etc.).

Librería	Módulo importado	Para qué se usa	¿Viene con Python?
PLY (Python Lex-Yacc)	ply.lex, ply.yacc	Genera el analizador léxico y el parser LALR(1) automáticamente	Sí
tkinter	tkinter, ttk, scrolledtext	Construye la interfaz gráfica de la aplicación	Sí (incluida)
sys	sys	Configurar la codificación UTF-8 de la salida estándar	Sí
os	os	Leer variables de entorno (en interpretador_ia.py)	Sí
io	io.StringIO	Capturar el stdout de cada fase para mostrarlo en la UI	Sí
contextlib	contextlib.redirect_stdout	Redirigir la salida estándar temporalmente	Sí
threading	threading.Thread	Ejecutar la síntesis de voz en un hilo separado para no congelar la UI	Sí
difflib	difflib.get_close_matches	Sugerir correcciones cuando el usuario escribe un token con error ortográfico	Sí
datetime	datetime.datetime	Registrar la hora de cada compilación en el historial	Sí
subprocess	subprocess.run	Ejecutar PowerShell para síntesis de voz (Windows)	Sí

¿Qué es PLY?

PLY (*Python Lex-Yacc*) es una implementación de las herramientas clásicas **lex** y **yacc** en Python. Permite definir:

- **lex**: reglas de expresión regular que convierten texto en tokens
- **yacc**: reglas gramaticales BNF que construyen el árbol de sintaxis

PLY genera automáticamente un parser tipo **LALR(1)** a partir de las funciones de Python que el programador define. El archivo `parsetab.py` es generado automáticamente por PLY al primera ejecución y contiene las tablas del parser.

3. Fase 1 — Análisis Léxico (lexer.py)

El analizador léxico es el primer componente del compilador. Su trabajo es leer la cadena de entrada carácter a carácter y agrupar los caracteres en unidades significativas llamadas **tokens**. Si encuentra algo que no reconoce, genera un error léxico (LEX-001, LEX-002 o LEX-003).

Alfabeto del lenguaje: 57 tokens en 6 categorías

El lenguaje define exactamente 57 tokens expresivos más operadores y contextos:

Categoría	Cantidad	Ejemplos
E1 · Sonidos vocales	11	mmm, ata, aah, uuh, oh, shh, uff, ay, ana, bah, pff
E2 · Gestos de manos	14	senalá, palma_arriba, toca, agita, apunta_si, mueve_pulgares...
E3 · Vocalizaciones	7	sonido_largo, sonido_corto, sonido_repetido, sonido_agudo...
E4 · Movimientos corporales	10	cabeza_si, cabeza_no, inclina_cuerpo, senala_propio...
E5 · Expresiones faciales	10	cierra_ojos, frunce_ceno, sonrie, llanto, boca_abierta...
E6 · Modificadores	5	rapido, lento, doble, triple, pausa
Operadores	5	+ ! ~ ;
Agrupación	2	()
Contexto	1	[manana] [tarde] [noche] [dolor]

Estructuras principales de lexer.py

token_map — diccionario de reconocimiento

Es un diccionario Python que mapea cada cadena del lenguaje a su tipo de token. Cuando PLY encuentra una palabra en la entrada, busca su minúscula en este diccionario para saber qué tipo de token es.

```
token_map = {
    'mmm': 'MMM', 'ata': 'ATA', 'sonrie': 'SONRIE',
    'palma_arriba': 'PALMA_ARRIBA', 'rapido': 'RAPIDO',
    # ... 57 entradas en total
}
```

t_TOKEN — función de reconocimiento principal

Es la función más importante del lexer. Cualquier secuencia de letras minúsculas y guiones bajos entra aquí. Si la palabra está en token_map, retorna el token correcto. Si no está, genera un error LEX-001 con sugerencia de corrección usando difflib.get_close_matches.

```
def t_TOKEN(t):
    r'[a-z][a-z0-9_]*'
    tipo = token_map.get(t.value)
    if tipo:
        t.type = tipo
        return t
    # Token no reconocido:
    suggs = difflib.get_close_matches(t.value, token_map.keys(), n=2, cutoff=0.6)
    print(f'[LEX-001] Token {t.value!r} no reconocido.')
    if suggs:
        print(f' → ¿Quisiste decir: {suggs}?')
```

t_CONTEXTO — reconocimiento de contextos

Detecta los corchetes [manana], [noche], etc. Valida que el contenido sea uno de los cuatro contextos válidos. Si no lo es, genera LEX-002 con sugerencia de corrección.

```
CONTEXTOS_VALIDOS = {'manana', 'noche', 'tarde', 'dolor'}
def t_CONTEXTO(t):
    r'\[[a-z]+\]'
    valor = t.value[1:-1] # quita los corchetes
    if valor not in CONTEXTOS_VALIDOS:
        print(f'[LEX-002] Contexto [{valor}] no válido.')
        return None # descartar el token
    t.value = valor
    return t
```

t_error — caracteres no válidos

Se ejecuta cuando PLY encuentra un carácter que ninguna otra regla puede consumir (por ejemplo @, {, ,). Genera LEX-003 y avanza un carácter con t.lexer.skip(1) para continuar el análisis.

```
def t_error(t):
    c = t.value[0]
    print(f'[LEX-003] Carácter {c!r} no válido.')
    t.lexer.skip(1)
```

Funciones para operadores y agrupación

Función	Regex	Token generado
t_MAS	\+	MAS — operador de secuencia
t_O	\	O — operador de alternativa
t_URGENTE	\!	URGENTE — marcador de urgencia (postfijo)
t_NEG	\~	NEG — negación (prefijo)
t_FIN_EXPR	\;	FIN_EXPR — separador de expresiones

t_LPAREN	\(	LPAREN — abre paréntesis de grupo
t_RPAREN	\)	RPAREN — cierra paréntesis de grupo

4. Fase 2 — Análisis Sintáctico (sintactico.py)

El analizador sintáctico recibe la lista de tokens del lexer y verifica que estén en el orden correcto según las **reglas gramaticales** del lenguaje. Si la secuencia es válida, construye un **AST** (Árbol de Sintaxis Abstracta). Si no, genera errores SIN-001 a SIN-010.

PLY implementa un parser **LALR(1)** (Look-Ahead LR con 1 token de anticipación), que es una técnica eficiente para parsear la mayoría de los lenguajes de programación.

Nodos del AST

El AST está formado por 5 clases de Python que representan cada nivel de la gramática:

Clase	Atributos	Representa
NodoPrograma	expresiones: list	La entrada completa (una o varias expresiones separadas por ';')
NodoExpresion	alternativas: list, contexto: str None	Una expresión con su contexto opcional y sus alternativas con ' '
NodoSecuencia	terminos: list	Una secuencia de términos unidos por '+'
NodoTermino	token_valor, token_tipo, negado, urgente, token_modificador	Un token modificado con sus modificadores (~, !)
NodoGrupo	expresion: NodoExpresion, negado, urgente	Una sub-expresión entre paréntesis: (A B)

Gramática del lenguaje (BNF)

```
programa : expresion
| programa ';' expresion
expresion : secuencia
| CONTEXTO secuencia
| expresion '|' secuencia
secuencia : termino
| secuencia '+' termino
termino : token_base
| '~' termino (negación)
| '!' termino (urgencia)
| '(' expresion ')' (agrupación)
| '(' expresion '!' ) (grupo urgente)
token_base: MMM | ATA | AAH | ... (57 tokens del alfabeto)
```

Tabla de precedencia de operadores

Define el orden en que se evalúan los operadores cuando no hay paréntesis. El último tiene mayor prioridad:

Prioridad	Tipo	Operador	Descripción
-----------	------	----------	-------------

1 (menor)	left	O ()	Alternativa: A B
2	left	MAS (+)	Secuencia: A + B
3	left	URGENTE (!)	Urgencia (postfijo): A!
4	right	NEG (~)	Negación (prefijo): ~A
5 (mayor)	left	LPAREN RPAREN ()	Agrupación: (A B)

Verificación de tipos en E6 (Regla semántica en sintáctico)

Los tokens E6 (*rapido, lento, doble, triple, pausa*) son **modificadores de intensidad**. El parser verifica reglas especiales para ellos:

Error	Situación	Ejemplo inválido	Correcto
SIN-007	Se negó un modificador E6	~rapido	~sonrie + lento
SIN-008	'!' aplicado a un modificador E6	lento !	sonrie !
SIN-009	Modificador E6 solo (sin token expresivo)	rapido	sonrie + rapido
SIN-010	Paréntesis sin cerrar o vacíos	(mmm uff	(mmm uff)

Función analizar()

Es el punto de entrada público del módulo. Recibe la cadena de texto y retorna el nodo raíz del AST (NodoPrograma) o None si hubo errores que impidieron la construcción:

```
def analizar(entrada):
    lexer.lineno = 1 # reiniciar contador de líneas
    resultado = parser.parse(entrada, lexer=lexer)
    return resultado # NodoPrograma o None
```

Función imprimir_ast()

Recorre el AST de forma recursiva e imprime una representación visual en árbol usando los caracteres ■■■ y ■■■. Se usa en la pestaña 'Fase 2 · Sintáctico' de la interfaz gráfica.

```
def imprimir_ast(nodo, prefijo='', es_ultimo=True):
    conector = '■■■ ' if es_ultimo else '■■■ '
    print(prefijo + conector + repr(nodo))
    # ... recursión para cada tipo de nodo
```

5. Fase 3 — Análisis Semántico (semantico.py)

El analizador semántico es la fase más compleja del compilador. Recibe el AST de la fase 2 y realiza tres tareas: (1) **valida** que la combinación de tokens tenga sentido semántico, (2) **clasifica** la secuencia en una de 8 estructuras definidas, y (3) **genera** la frase en español natural.

Tabla de símbolos (TablaSimbolos)

Clase simple que mantiene el estado global durante la compilación. Guarda el contexto activo y si hubo urgencia:

```
class TablaSimbolos:
    def reset(self):
        self.contexto_activo = None # 'manana', 'tarde', 'noche', 'dolor'
        self.hay_urgencia = False
        tabla = TablaSimbolos() # instancia global, una sola por compilación
```

Tablas de datos semánticos

El módulo contiene tres diccionarios estáticos que definen el significado de los tokens:

- **CATEGORIA**: mapea cada tipo de token a su categoría E1–E6
- **BASE**: significado por defecto de cada token en español
- **OVERRIDE**: significados especiales según el contexto activo

Ejemplo de cómo funciona el override:

```
# Sin contexto:
boca_abierta → 'tiene hambre o sed'

# Con contexto [manana]:
boca_abierta → 'quiere desayunar'

# Con contexto [noche]:
boca_abierta → 'quiere cenar'
```

Estructuras Semánticas Definidas

Esta es la aportación principal al análisis semántico. Se definen **8 estructuras** de comunicación válidas. Una secuencia pertenece a una estructura si contiene al menos un *token núcleo* de esa estructura (activo, no negado, no E6):

Estructura	Tokens núcleo (ejemplos)	Descripción
PETICION	palma_arriba, senala, toca, mira_objeto, pun	Solicitud o petición de algo
EXPRESION_DOLOR	ay, uff, uuh, frunce_ceno, sonido_agudo, llan	Comunicación de dolor o malestar físico

EMOCION_POSITIVA	sonrie, aah, apunta_si, acerca_cuerpo	Estado emocional positivo o de acuerdo
EMOCION_NEGATIVA	llanto, bah, pff, mira_abajo, aleja_cuerpo	Estado emocional negativo o rechazo
LLAMADO_ATENCION	agita, levanta_brazo, ata, ana, sonido_largo	Llamado de atención o solicitud de ayuda
NECESIDAD_FISIOLOGICA	boca_abierta, shh, sonido_ronquido, cierra_ojos	Necesidad física básica (hambre, baño, sueño)
CONFIRMACION_RESPUESTA	cabeza_si, cabeza_no, cabeza_lado, apunta_si	Respuesta afirmativa, negativa o de duda
INDECISION_CONFUSION	mmm, encoge_hombros, separa_manos, mira_adelante	Estado de duda o confusión

Función clasificar_estructura()

Recibe el conjunto de tokens activos (no negados, no E6) y retorna el nombre de la primera estructura cuyo conjunto núcleo tenga intersección con la secuencia. Si ninguna coincide, retorna None y se genera el error SEM-005.

```
def clasificar_estructura(tipos_activos: set):
    for nombre, estructura in ESTRUCTURAS_SEMANTICAS.items():
        if estructura['tokens_nucleo'] & tipos_activos:
            return nombre # primera estructura que coincide
    return None # → genera SEM-005
```

Detección de contradicciones (SEM-006)

El sistema detecta 4 pares de combinaciones contradictorias. Si la secuencia activa tiene tokens de ambos grupos de un par, se genera SEM-006:

Grupo A	Grupo B	Contradicción
sonrie, aah, acerca_cuerpo	llanto, frunce_ceno, bah, aleja_cuerpo	Positivo + Negativo/Dolor
cabeza_si, apunta_si	cabeza_no, bah, pff	Afirmación + Negación
acerca_cuerpo	aleja_cuerpo	Acercarse + Alejarse
sonido_suave	sonido_agudo, sonido_repetido, sonido_largo	Calma + Alerta/Insistencia

Función detectar_patron()

Esta función complementa a clasificar_estructura. Determina el patrón semántico dominante de la secuencia para seleccionar la plantilla de frase más adecuada. Usa los conjuntos TOKENS_DOLOR, TOKENS_URGENCIA, TOKENS_PETICION, etc.:

Patrón detectado	Condición principal
dolor_urgente	hay '!' o TOKENS_URGENCIA + TOKENS_DOLOR o contexto [dolor]
urgencia	hay '!' o TOKENS_URGENCIA activos

dolor	contexto [dolor] o TOKENS_DOLOR activos
emocional_positivo	TOKENS_POSITIVO activos (sonrie, aah, apunta_si)
hambre_ctx	BOCA_ABIERTA + contexto temporal
sueno_noche	TOKENS_CANSANCIO + contexto [noche]
confirmacion	mayormente TOKENS_CONFIRM activos
peticion	TOKENS_PETICION activos
general	ninguno de los anteriores → SEM-001

Función compilar() — punto de entrada

Es la función que llama la interfaz gráfica. Recibe la cadena de entrada, ejecuta el análisis completo y retorna la lista de frases generadas:

```
def compilar(entrada):
    tabla.reset() # limpiar estado
    ast = analizar(entrada) # fase 1 + 2
    if not ast:
        return ['Error: no se pudo analizar la entrada.']
    frases = []
    for expresion in ast.expresiones:
        frase = interpretar_expresion(expresion)
        frases.append(frase)
    return frases
```

Tabla completa de errores semánticos

Código	Tipo	Descripción	Ejemplo que lo activa
SEM-001	Advertencia	Combinación sin patrón reconocible	oh + abre_ojos (tokens de sorpresa sin estructura clara)
SEM-002	Advertencia	Todos los tokens están negados (~)	~sonrie + ~cabeza_si
SEM-003	Advertencia	'pausa' sola sin ningún otro token	pausa
SEM-004	Advertencia	Secuencia mayor de 6 tokens	a + b + c + d + e + f + g
SEM-005	Advertencia	No pertenece a ninguna estructura definida	oh + abre_ojos (ningún token núcleo coincide)
SEM-006	Advertencia	Combinación semánticamente contradictoria	sonrie + llanto (positivo + negativo)

6. Interfaz Gráfica (interfaz.py)

La interfaz gráfica está construida con **Tkinter**, la librería estándar de Python para GUIs. Permite al cuidador ingresar expresiones de forma visual sin necesidad de conocer los tokens de memoria.

Componentes de la UI

Componente	Widget Tkinter	Función
Panel izquierdo de tokens	Canvas + Frame scrollable	Muestra todos los 57 tokens organizados por categoría como botones
Selector de contexto	Radiobutton (×4) + Button	Permite seleccionar [manana], [tarde], [noche] o [dolor]
Campo de expresión	Text (height=2)	Área donde se escribe o construye la expresión
Botones de operadores	Button (×5)	Insertan + ! ~ ; en la expresión con un clic
Botón Compilar	Button	Ejecuta las 3 fases del compilador
Área de traducción	Text (readonly)	Muestra la frase resultante en amarillo prominente
Botón ■	Button	Lee la frase en voz alta (PowerShell + System.Speech en Windows)
Pestaña Fase 1 · Léxico	ScrolledText	Muestra los tokens generados y errores LEX-###
Pestaña Fase 2 · Sintáctico	ScrolledText	Muestra el AST y errores SIN-###
Pestaña Fase 3 · Semántico	ScrolledText	Muestra la frase generada, estructuras y advertencias SEM-###
Pestaña Historial	ScrolledText	Registro de todas las compilaciones de la sesión con hora
Pestaña ■ Errores	ScrolledText (readonly)	Tabla estática con todos los códigos de error documentados

Flujo de compilación en _compilar()

Cuando el usuario presiona '■ Compilar', la función _compilar() ejecuta:

- 1. Construye la cadena de entrada con _construir_entrada() (añade contexto si hay radio button seleccionado)
- 2. **Fase 1:** Captura stdout de lexer, genera tabla de tokens
- 3. **Fase 2:** Captura stdout de analizar(), imprime el AST
- 4. **Fase 3:** Llama a compilar(), muestra la frase y advertencias
- 5. Si hubo SIN-007/008/009/010, cambia automáticamente a la pestaña Semántico
- 6. Agrega la compilación al historial con la hora actual

Cómo se captura el stdout de cada fase

Como PLY y el compilador imprimen sus mensajes con `print()`, la interfaz usa `contextlib.redirect_stdout()` con un buffer `io.StringIO()` para capturarlos y mostrarlos en las pestañas:

```
buf = io.StringIO()
with contextlib.redirect_stdout(buf):
    lexer.input(entrada)
    toks = list(lexer)
errores_lexico = buf.getvalue() # todo lo que imprimió el lexer
```

7. Tabla Completa de Códigos de Error

El compilador genera 16 códigos de error/advertencia distintos distribuidos en las 3 fases:

Fase 1 — Errores Léxicos (LEX-###)

Código	Descripción	Ejemplo de entrada	Mensaje de ayuda
LEX-001	Token no reconocido	sonri	¿Quisiste decir: sonrie?
LEX-002	Contexto inválido entre corchetes [manyana]		¿Quisiste decir: [manana]?
LEX-003	Carácter ilegal fuera del alfabeto	mmm @ sonrie	solo se permiten letras minúsculas, + !~;[]

Fase 2 — Errores Sintácticos (SIN-###)

Código	Descripción	Ejemplo
SIN-001	Operador + o en posición inválida	+ mmm / mmm + + palma_arriba
SIN-002	! sin token previo / ~ mal colocado	! sonido_largo
SIN-003	Token faltante después de +	mmm +
SIN-004	; sin expresión válida antes o después	; sonrie
SIN-005	Contexto [...] fuera del inicio	sonrie [dolor] uff
SIN-006	La entrada terminó de forma inesperada	mmm +
SIN-007	Se intentó negar un modificador E6	~rapido
SIN-008	! aplicado a un modificador E6	lento !
SIN-009	Modificador E6 solo sin token expresivo	rapido
SIN-010	Paréntesis sin cerrar o paréntesis vacíos	(mmm uff /)

Fase 3 — Advertencias Semánticas (SEM-###)

Código	Descripción	Ejemplo
SEM-001	Combinación sin patrón reconocible	oh + abre_ojos
SEM-002	Todos los tokens están negados (~)	~sonrie + ~cabeza_si
SEM-003	'pausa' sola sin ningún otro token	pausa
SEM-004	Secuencia demasiado larga (>6 tokens)	a+b+c+d+e+f+g

SEM-005	No pertenece a ninguna estructura semántica	oh + abre_ojos (sin núcleo)
SEM-006	Combinación semánticamente contradictoria	sonrie + llanto

Nota: los errores LEX y SIN impiden que el compilador avance a la siguiente fase. Los SEM son *advertencias*: el compilador genera la frase de todas formas, pero avisa al cuidador que la combinación puede ser ambigua o contradictoria.

8. Ejemplo Completo Paso a Paso

Para ilustrar el funcionamiento del compilador, se traza la entrada:

[dolor] ay + senala_propio + levanta_brazo !

Paso 1 — Análisis Léxico

El lexer escanea la cadena y produce los siguientes tokens:

#	Tipo de token	Categoría	Valor original	Significado semántico
1	CONTEXTO	Contexto	[dolor]	Contexto: situación de dolor
2	AY	E1 · Vocal	ay	Sonido vocal de dolor agudo
3	MAS	Operador	+	Operador de secuencia
4	SENALA_PROPIO	E4 · Corporal	senala_propio	Señala hacia sí mismo
5	MAS	Operador	+	Operador de secuencia
6	LEVANTA_BRAZO	E4 · Corporal	levanta_brazo	Levanta el brazo pidiendo ayuda
7	URGENTE	Operador	!	Marcador de urgencia máxima

✓ Sin errores léxicos. Se generaron 7 tokens válidos.

Paso 2 — Análisis Sintáctico

El parser verifica que los tokens estén en orden gramatical correcto y construye el AST:

```
■■■ Programa(1 expresion(es))
■■■ Expresion([dolor] 1 alternativa(s))
■■■ Secuencia(3 termino(s))
■■■ Termino(ay)
■■■ Termino(senala_propio)
■■■ Termino(levanta_brazo!)
```

El parser verifica:

- ✓ **CONTEXTO** → **secuencia**: contexto al inicio de la expresión (válido)
- ✓ **secuencia MAS termino**: los '+' conectan términos correctamente
- ✓ **termino URGENTE**: '!' aplicado a 'levanta_brazo' (no es E6, válido)
- ✓ **ay, senala_propio, levanta_brazo**: todos son tokens del alfabeto

✓ Sin errores sintácticos. AST construido exitosamente.

Paso 3 — Análisis Semántico

El analizador semántico recorre el AST nodo a nodo:

3.1 — Obtener significados (con override de contexto [dolor])

Token	Significado base	Override [dolor]	Significado final
ay	siente dolor agudo o se asustó	siente un dolor muy intenso	siente un dolor muy intenso
senala_propio	es para él o le afecta a él	le duele algo en su cuerpo	le duele algo en su cuerpo
levanta_brazo	pide ayuda o atención	necesita atención o medicina	necesita atención o medicina ¡URGENTE!

3.2 — Clasificación de estructura semántica

El sistema construye el conjunto de tokens activos (no negados, no E6):

```
tipos_activos = {'AY', 'SENALA_PROPIO', 'LEVANTA_BRAZO'}
```

Busca en ESTRUCTURAS_SEMANTICAS:

Estructura probada	Intersección con tipos_activos	¿Coincide?
PETICION	SENALA_PROPIO no está en PETICION $\rightarrow \emptyset$	$\times$
EXPRESION_DOLOR	AY $\in$ tokens_nucleo $\rightarrow \{AY\}$	✓ — primera coincidencia
LLAMADO_ATENCION	(no se llega, ya encontró EXPRESION_DOLOR)	—

→ Estructura clasificada: **EXPRESION_DOLOR**

3.3 — Detección de contradicciones

Se comparan los tokens activos contra los 4 pares de contradicción. {'AY','SENALA_PROPIO','LEVANTA_BRAZO'} no tiene tokens de grupos opuestos → **sin contradicción**.

3.4 — Advertencias semánticas

El sistema evalúa cada advertencia:

Advertencia	Condición	¿Se activa?
SEM-001	patrón == 'general'	No — patrón es 'dolor_urgente'
SEM-002	todos los tokens negados	No — ninguno está negado
SEM-003	tipos == ['PAUSA']	No
SEM-004	más de 6 tokens	No — hay 3 tokens
SEM-005	clasificar_estructura() == None	No — se clasificó como EXPRESION_DOLOR
SEM-006	detectar_contradicción() != None	No — sin contradicción

✓ Sin advertencias semánticas.

3.5 — Generación de la frase

Con patrón 'dolor_urgente' y hay_urgente=True:

```
def generar_frase(infos, patron, contexto, hay_urgente):  
    # patron == 'dolor_urgente'  
    base = _frase_dolor(infos, contexto)  
    # → 'SEÑALA_PROPIO' en tipos → 'Le duele algo en su cuerpo y pide ayuda'  
    return f'¡{base}! Necesita atención urgente ahora mismo.'
```

El contexto [dolor] añade el prefijo:

```
frase = 'Con señales de dolor: ¡le duele algo en su cuerpo y pide ayuda!'  
' Necesita atención urgente ahora mismo.'
```

Con señales de dolor: ¡Le duele algo en su cuerpo y pide ayuda! Necesita atención urgente ahora mismo.

9. Resumen de Todas las Funciones

Referencia rápida de cada función pública del compilador:

lexer.py

Función / Variable	Tipo	Descripción
tokens	tuple	Define los 66 tipos de token que PLY debe reconocer
token_map	dict	Mapea cadenas de texto a tipos de token
t_COMMENT	función	Descarta líneas que empiezan con # (comentarios)
t_CONTEXTO	función	Reconoce y valida [manana], [tarde], [noche], [dolor]
t_MAS ... t_RPAREN	función×7	Una por operador/agrupación, retornan su token
t_newline	función	Cuenta saltos de línea para reportar número de línea en errores
t_TOKEN	función	Reconoce cualquier identificador; valida contra token_map
t_error	función	Caracteres ilegales: genera LEX-003 y avanza
_columna(t)	función	Calcula la columna del token para el mensaje de error
lexer	objeto	Instancia del analizador léxico, lista para usar

sintactico.py

Función / Clase	Tipo	Descripción
NodoPrograma	clase	Nodo raíz del AST: contiene todas las expresiones
NodoExpresion	clase	Una expresión con contexto y alternativas
NodoSecuencia	clase	Una secuencia de términos unidos por +
NodoTermino	clase	Un token con flags: negado, urgente, es_modificador
NodoGrupo	clase	Sub-expresión entre paréntesis (A B)
_MODIFICADORES_E6	frozenset	Constante con los 5 tokens E6 para verificación de tipo
p_programa...	funciones PLY	Reglas gramaticales; cada una corresponde a una producción BNF
p_error	función PLY	Recuperación de errores: clasifica el error y llama error()
imprimir_ast	función	Imprime el árbol AST de forma visual con caracteres de árbol
analizar	función	Punto de entrada: recibe string, retorna NodoPrograma o None

semantico.py

Función / Variable	Tipo	Descripción
TablaSimbolos	clase	Estado global de la compilación (contexto, urgencia)
CATEGORIA	dict	Mapea tipo de token a su categoría E1–E6
BASE	dict	Significado por defecto de cada token
OVERRIDE	dict	Significados alternativos según contexto activo
TOKENS_DOLOR/URGENCIA/...	frozenset×8	Grupos de tokens por función semántica
ESTRUCTURAS_SEMANTICAS	dict	Las 8 estructuras de comunicación con sus tokens núcleo
_PARES_CONTRADICTORIOS	list	Pares de grupos incompatibles para detectar SEM-006
clasificar_estructura()	función	Retorna la estructura a la que pertenece la secuencia
detectar_contradicion()	función	Retorna descripción si hay tokens contradictorios
significado_token()	función	Retorna el significado con override de contexto aplicado
detectar_patron()	función	Detecta el patrón dominante para seleccionar la plantilla
generar_frase()	función	Genera la frase en español según el patrón detectado
interpretar_secuencia()	función	Procesa una NodoSecuencia: infos + validaciones + frase
interpretar_expresion()	función	Procesa un NodoExpresion: alternativas + prefijo contexto
compilar()	función	Punto de entrada público: recibe string, retorna list[str]

10. Conclusión

El compilador implementa de forma completa las **tres fases clásicas** de un compilador: análisis léxico, análisis sintáctico y análisis semántico.

La fase semántica va más allá de un compilador básico: no solo verifica tipos sino que **clasifica el mensaje en una de 8 estructuras comunicativas**, detecta **contradicciones semánticas** y **genera lenguaje natural** adaptado al contexto del momento del día o estado físico.

Aspecto	Implementación
Tokens del alfabeto	57 tokens en 6 categorías (E1–E6)
Herramienta de análisis	PLY (Python Lex-Yacc) — LALR(1)
Errores detectados	16 códigos: 3 léxicos, 10 sintácticos, 6 semánticos
Estructuras semánticas	8 categorías de comunicación formalmente definidas
Detección de contradicciones	4 pares de grupos incompatibles
Nodos del AST	5 clases: Programa, Expresión, Secuencia, Término, Grupo
Interfaz	Tkinter con 5 pestañas, historial y síntesis de voz
Líneas de código totales	~700 entre los 4 archivos principales

Este compilador es una herramienta funcional y real: fue diseñado para una persona específica, con tokens derivados de señales reales observadas, y produce interpretaciones que el cuidador puede entender inmediatamente.